

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.5.6

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P3375 KMP	普及+/提高	KMP、前缀函数、字符串匹配
P5734 文字处理软件	普及-	字符串模拟、子串操作
B2106 矩阵转置	入门	二维数组、矩阵遍历
P2615 神奇的幻方	普及-	模拟、二维数组

（一）KMP

题目信息

题目链接：<https://www.luogu.com.cn/problem/P3375>

知识点：KMP、前缀函数、字符串匹配

题目简述

给出主串 s_1 和模式串 s_2 ，要求找出 s_2 在 s_1 中所有出现的位置，并按从小到大的顺序输出。

除了匹配位置以外，还需要输出模式串每一个前缀的最长 border 长度。这里的 border 指既是前缀又是后缀、且不能等于整个串本身的子串。

对于全部数据，保证 $1 \leq |s_1|, |s_2| \leq 10^6$ ，且字符串只由大写英文字母组成。

1.00s, 512MB。

题目分析

1. 读题

题目实际上包含两个目标：

1. 在长串中找到模式串的所有出现位置；
2. 对模式串的每个前缀，求其最长 border 长度。

这两个任务看起来分开，但本质都和“前后缀匹配信息”有关。

2. 性质观察

如果用普通暴力匹配，每次失配都让模式串重新回到开头，那么在 10^6 级别数据下会超时。题目的关键就在于失配之后，不应把已经比较过的信息全部丢掉，而是利用模式串自身的前后缀重合关系继续匹配。

而“每个前缀的最长 border 长度”本身，正好就是 KMP 中 `next` 数组或前缀函数所维护的信息。

3. 程序设计思路

整体可以分两步：

1. 先对模式串构造前缀函数，得到每个位置对应的最长相等前后缀长度；
2. 再用这份前缀函数去扫描主串；
3. 若当前字符匹配，就同时向后推进；
4. 若失配，就按照前缀函数回退模式串指针，而不是让主串回头；
5. 每当模式串全部匹配完成，就记录一个出现位置，并继续按照前缀函数寻找下一次出现。

最后，前缀函数数组本身就可以直接作为题目要求的 border 长度输出。

4. 数据结构与算法选择

本题不需要复杂容器，核心是字符串数组加 KMP 算法。KMP 的优势在于把大量重复比较压缩掉，使得匹配过程保持线性复杂度，同时也能自然得到每个前缀的 border 信息。

5. 时空复杂度分析

- 时间复杂度： $O(|s_1| + |s_2|)$ ，构造前缀函数和匹配过程都只需线性扫描。
- 空间复杂度： $O(|s_2|)$ ，主要用于存储模式串的前缀函数数组。

这正是本题在大规模字符串限制下的标准做法。

代码实现

记录详情

编程语言

C O2

代码长度

1.03KB

用时

42ms

内存

3.11MB

测试点信息

源代码

测试点信息

Subtask #0

#2

AC

3ms/812.00KB

#3

AC

3ms/788.00KB

#4

AC

3ms/820.00KB

Subtask #1

#5

AC

3ms/788.00KB

#6

AC

3ms/808.00KB

#7

AC

3ms/900.00KB

#8

AC

3ms/788.00KB

Subtask #2

#9

AC

7ms/1.31MB

#10

AC

6ms/1.27MB

#11

AC

7ms/1.25MB

#12

AC

59ms/1.25MB

#13

AC

9ms/1.27MB

#14

AC

55ms/1.43MB

Subtask #3

#1

AC

30ms/3.11MB

jsj0708Baijiahui

所属题目

P3375 【模板】KMP

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:27:59

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define MAXN 1000005
void buildNext(char* pattern, int* next, int len)
{
    next[0] = 0;
    int prefix_len = 0;
    int i = 1;
    while (i < len)
    {
        if (pattern[prefix_len] == pattern[i])
        {
            prefix_len++;
            next[i] = prefix_len;
            i++;
        }
        else
        {
            if (prefix_len == 0)
            {
                next[i] = 0;
                i++;
            }
            else
            {
                prefix_len = next[prefix_len - 1];
            }
        }
    }
}

void KMP_Search(char* text, char* pattern, int* next, int m)
{
    int n = strlen(text);
    int i = 0;
    int j = 0;
    while (i < n)
    {
        if (text[i] == pattern[j])
        {
            i++;
            j++;
        }
    }
}

```

```

        else
        {
            if (j > 0)
                j = next[j - 1];
            else
                i++;
        }
        if (j == m)
        {
            printf("%d\n", i - j + 1);
            j = next[j - 1];
        }
    }
}

char s1[MAXN], s2[MAXN];
int main()
{
    scanf("%s%s", s1, s2);
    int n = strlen(s1);
    int m = strlen(s2);
    int* next = (int*)calloc(m, sizeof(int));
    buildNext(s2, next, m);
    KMP_Search(s1, s2, next, m);
    for (int i = 0; i < m; i++)
        printf("%d ", next[i]);
    printf("\n");
    free(next);
    return 0;
}

```

(二) 文字处理软件

题目信息

题目链接：<https://www.luogu.com.cn/problem/P5734>

知识点：字符串模拟、子串操作

题目简述

题目给出一个初始字符串，之后需要依次执行若干次文字处理操作。操作包括在末尾追加字符串、截取一段子串、在指定位置插入字符串，以及查找某个子串第一次出现的位置。

对于操作 1、2、3，都需要输出处理后的整个字符串；对于操作 4，需要输出查找结果。

数据保证操作次数 $q \leq 100$ ，初始字符串长度不超过 100。

1.00s, 128MB。

题目分析

1. 读题

这是一道典型的字符串模拟题。题目中所有操作都直接围绕“当前文档内容”展开，每一步的结果都会影响下一步。

2. 性质观察

虽然操作看起来不少，但数据范围非常小，字符串长度和操作次数都不大。因此完全没必要引入复杂数据结构，直接按题意一条条模拟即可。

四类操作本质上分别对应：

1. 字符串拼接；
2. 子串截取；
3. 中间插入；
4. 子串查找。

3. 程序设计思路

维护一个当前字符串 s ：

1. 若操作为 1 str ，就把 str 拼接到 s 末尾；
2. 若操作为 2 $a\ b$ ，就把 s 替换为从位置 a 开始、长度为 b 的那一段；
3. 若操作为 3 $a\ str$ ，就把 str 插入到 s 的第 a 个字符之前；
4. 若操作为 4 str ，就在 s 中顺序查找 str 最早出现的位置，找不到则输出 -1 。

每次完成操作后，按题意输出对应结果即可。

4. 数据结构与算法选择

本题最合适的就是直接使用字符串。因为题目本身就是在做文本编辑，且数据规模很小，字符串自带的拼接、截取和查找能力已经足够完成全部操作。

5. 时空复杂度分析

- 时间复杂度：单次操作最多为 $O(n)$ ，总复杂度在本题数据范围下完全可接受。
- 空间复杂度： $O(n)$ ，主要用于维护当前字符串内容。

由于字符串长度上限很低，直接模拟是最稳妥的方案。

代码实现

首页 / 评测记录 / 评测详情

R281460598 记录详情

编程语言

C++2

代码长度

1.15KB

用时

17ms

内存

836.00KB

测试点信息

源代码

测试点信息

#1

AC

4ms/788.00KB

#2

AC

3ms/788.00KB

#3

AC

3ms/836.00KB

#4

AC

4ms/812.00KB

#5

AC

3ms/812.00KB

jsj0708Baijiahui

所属题目

P5734 【深基6.例6】文字处理...

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:28:58

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define max 100010
void op1(char str[])
{
    char temp1[max] = { 0 };
    scanf("%s", temp1);
    strcat(str, temp1);
    printf("%s\n", str);
}
void op2(char str[])
{
    int k = 0;
    int m = 0;
    scanf(" %d %d", &k, &m);
    for (int i = k; i < k + m; i++)
    {
        str[i - k] = str[i];
    }
    str[m] = '\0';
    printf("%s\n",str);
}
void op3(char str[])
{
    int k = 0;
    scanf("%d", &k);
    char temp[max] = { 0 };
    scanf("%s", temp);
    int sz1 = strlen(str);
    int sz2 = strlen(temp);
    for (int i = sz1; i >= k; i--)
    {
        str[i + sz2] = str[i];
    }
    int num = 0;
    for (int i = k; i < k + sz2; i++)
    {
        str[i] = temp[num++];
    }
    printf("%s\n", str);
}
void op4(char str[])

```

```

{
    char temp[max] = { 0 };
    scanf("%s", temp);
    char* dest = strstr(str, temp);
    if (dest == NULL)
    {
        printf("-1\n");
    }
    else
    {
        printf("%d\n", dest - str);
    }
}

int main()
{
    int q = 0;
    scanf("%d", &q);
    char str[max] = { 0 };
    scanf("%s", str);
    while (q--)
    {
        int n = 0;
        scanf("%d", &n);
        switch(n)
        {
            case 1:
                op1(str);
                break;
            case 2:
                op2(str);
                break;
            case 3:
                op3(str);
                break;
            case 4:
                op4(str);
                break;
        }
    }
    return 0;
}

```

(三) 矩阵转置

题目信息

题目链接：<https://www.luogu.com.cn/problem/B2106>

知识点：二维数组、矩阵遍历

题目简述

输入一个 n 行 m 列的矩阵 A ，要求输出它的转置矩阵 A^T 。

所谓转置，就是把原矩阵的行列互换：原来的第 i 行第 j 列，变成新矩阵的第 j 行第 i 列。

1.00s, 128MB。

题目分析

1. 读题

本题没有复杂的运算，核心只是把矩阵中元素的位置交换为“行列互换”的形式，再按新的顺序输出。

2. 性质观察

若原矩阵大小为 $n \times m$ ，那么转置后矩阵大小就变成 $m \times n$ 。原来位置为 (i, j) 的元素，在输出时应当出现在 (j, i) 。

3. 程序设计思路

先用二维数组读入原矩阵，然后按转置矩阵的行列顺序输出：

1. 外层循环遍历原矩阵的列；
2. 内层循环遍历原矩阵的行；
3. 每次输出 $a[j][i]$ ，就相当于输出转置后的第 i 行第 j 列元素。

4. 数据结构与算法选择

本题只需要一个普通二维数组存储原矩阵即可。算法上采用直接读入加直接按转置顺序输出，没有任何额外技巧。

5. 时空复杂度分析

- 时间复杂度： $O(nm)$ ，每个元素读入一次、输出一次。
- 空间复杂度： $O(nm)$ ，需要保存整个原矩阵。

这是矩阵转置最直接、也最清晰的实现方式。

代码实现

首页 / 评测记录 / 评测详情

R281460665 记录详情

编程语言C O2

代码长度381B

用时17ms

内存1.04MB

测试点信息源代码

测试点信息

#1AC3ms/788.00KB

#2AC3ms/788.00KB

#3AC4ms/820.00KB

#4AC4ms/812.00KB

#5AC3ms/1.04MB

jsj0708Baijiahui

所属题目B2106 矩阵转置

评测状态Accepted

评测分数100

提交时间2026-06-11 17:29:28

```
#include<stdio.h>
#include <stdlib.h>
int main()
{
    int n,m;
    scanf("%d %d",&n,&m);
    int arr[100][100];
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<m;j++)
        {
            scanf("%d",&arr[i][j]);
        }
    }
    for(int i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
        {
            printf("%d ",arr[j][i]);
        }
        printf("\n");
    }
}
```

(四) 神奇的幻方

题目信息

题目链接：<https://www.luogu.com.cn/problem/P2615>

知识点：模拟、二维数组

题目简述

给定一个奇数 N ，需要按照题目指定的规则，构造一个 $N \times N$ 的幻方。幻方中填入 1 到 N^2 ，并且每行、每列以及两条对角线上的数字和都相同。

构造规则已经在题面中给出：先把 1 放在第一行正中间，之后每次根据当前位置是否到达边界、右上角是否已被占用，决定下一个数应该放在哪里。

对于全部数据， $1 \leq N \leq 39$ ，且 N 为奇数。

1.00s, 128MB。

题目分析

1. 读题

题目并不是让我们自己设计一种构造方法，而是已经明确给出了幻方的填写规则。也就是说，只需要严格按照规则模拟填数过程即可。

2. 性质观察

每个数字的位置都只和前一个数字的位置有关，因此整个过程是一个典型的逐步模拟问题。由于 N 最大只有 39，二维数组直接维护整张幻方完全没有压力。

3. 程序设计思路

先把 1 放在第一行中间位置，然后从 2 一直填到 N^2 ：

1. 先尝试把下一个数放到当前格子的右上方；
2. 如果当前位置在第一行，就向上越界后回到最后一行；
3. 如果当前位置在最后一列，就向右越界后回到第一列；
4. 如果右上方那个格子已经填过数，则改为放到当前格子的正下方；
5. 按此规则不断更新当前位置，直到全部数字填完。

4. 数据结构与算法选择

本题适合使用二维数组直接记录每个位置上的数字。算法上采用纯模拟即可，因为状态转移明确、规模也很小，不需要任何复杂优化。

5. 时空复杂度分析

- 时间复杂度： $O(N^2)$ ，需要依次放置从 1 到 N^2 的所有数字。
- 空间复杂度： $O(N^2)$ ，使用一个二维数组保存整个幻方。

在题目给定的范围下，这样的复杂度非常宽松。

首页 / 评测记录 / 评测详情

R281460745 记录详情

编程语言

C++

代码长度

1.27KB

用时

33ms

内存

864.00KB

测试点信息

源代码

测试点信息

#1 AC 3ms/816.00KB	#2 AC 3ms/816.00KB	#3 AC 4ms/812.00KB	#4 AC 3ms/816.00KB	#5 AC 3ms/816.00KB	#6 AC 4ms/864.00KB	#7 AC 3ms/784.00KB
#8 AC 3ms/816.00KB	#9 AC 4ms/816.00KB	#10 AC 3ms/816.00KB				

jsj0708Baijiahui

所属题目

P2615 [NOIP 2015 提高组] 神奇...

评测状态

Accepted

评测分数

100

提交时间

2026-06-11 17:30:06

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define max 100010
int main()
{
    int N = 0;
    scanf("%d", &N);
    int** arr = (int**)malloc(N * sizeof(int*));
    for (int i = 0; i < N; i++)
    {
        arr[i] = (int*)malloc(N * sizeof(int));
    }

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            arr[i][j] = 0;
        }
    }

    arr[0][N / 2] = 1;
    int num = 2;
    int temp1 = 0;
    int temp2 = N/2;
    while (num <= N * N)
    {
        if (temp1 == 0 && temp2 != N - 1)
        {
            arr[N - 1][temp2 + 1]=num;
            temp1 = N - 1;
            temp2 = temp2 + 1;
            num++;
        }
        else if (temp1 != 0 && temp2 == N - 1)
        {
            arr[temp1 - 1][0] = num;
            temp1 = temp1 - 1;
            temp2 = 0;
            num++;
        }
        else if (temp1 == 0 && temp2 == N - 1)

```

```

        {
            arr[temp1 + 1][temp2] = num;
            temp1 = temp1 + 1;
            temp2 = temp2;
            num++;
        }
    else if (temp1 != 0 && temp2 != N - 1)
    {
        if (arr[temp1 - 1][temp2 + 1] == 0)
        {
            arr[temp1 - 1][temp2 + 1] = num;
            temp1 = temp1 - 1;
            temp2 = temp2 + 1;
            num++;
        }
        else
        {
            arr[temp1 + 1][temp2] = num;
            temp1 = temp1 + 1;
            temp2 = temp2;
            num++;
        }
    }
}

for (int i = 0; i < N; i++)
{
    for (int j = 0; j < N; j++)
    {
        printf("%d ", arr[i][j]);
    }
    printf("\n");
}

for (int i = 0; i < N; i++)
{
    free(arr[i]);
}
free(arr);
return 0;
}

```